

Programação Orientada a Objetos

OBJETOS DINÂMICOS

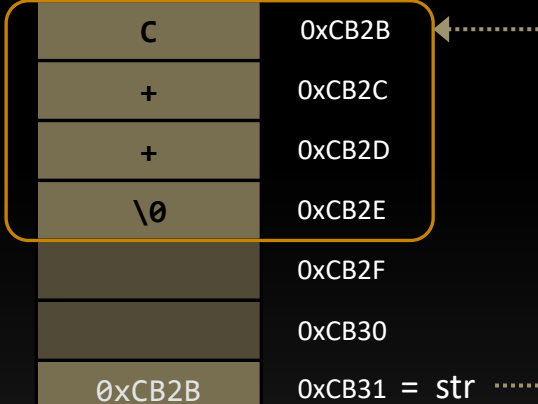
em

C++

Introdução

- É preciso ter **cuidado** com alocação dinâmica
 - **new** e **delete** devem ser casados
 - Usando construtor e destrutor
 - **new** e **delete** devem ser compatíveis
 - Mesmo em múltiplos construtores
 - Os [] fazem diferença
 - **Cópias profundas** são necessárias
 - Construtor de cópia
 - Atribuição

Memória	NEW
C	0xCB2B
+	0xCB2C
+	0xCB2D
\0	0xCB2E
	0xCB2F
	0xCB30
0xCB2B	0xCB31 = str



Introdução

- As **classes abstraem a alocação dinâmica**
 - Não importa como a classe lida com a memória
 - A interface pública provê funcionalidades

```
class String
{
    ...

public:
    String();                // construtor padrão
    ~String();              // destrutor
    String(const char s[]);  // construtor
    String(const String& s);  // construtor de cópia
    String & operator=(const String & s); // operador de atribuição
};
```

Classes e Tipos

- As **classes** podem ser **usadas como tipos**
 - Em programas ou **novas classes**

```
class String
{
private:
    char * str;
    size_t tam;

    ...
};
```

```
#include "String.h"
#include <string>
using std::string;

class Artigo
{
private:
    String titulo;
    string autor;

    ...
};
```

String e string realizam
alocação dinâmica de
memória

Artigo precisa de um
construtor de cópia e
operador de atribuição?

Classes e Tipos

- A classe **Artigo** não faz **alocação dinâmica**
 - Não faz uso de **new** diretamente
 - Não precisa fazer cópias profundas
 - Pode usar os métodos padrões

```
class Artigo
{
private:
    String titulo;
    string autor;
    ...
};
```

```
int main()
{
    Artigo gasolina { "Combustão", "Nikolaus Otto" };
    Artigo eletrico { "Elétrico", "Ányos Jedlik" };

    Artigo carro { gasolina };
    carro = eletrico;
}
```

Classes e Tipos

- As **implementações do compilador** funcionam
 - Construtor de cópia
 - Operador de atribuição

```
Artigo gasolina { "Combustão", "Nikolaus Otto" };  
Artigo eletrico { "Elétrico", "Ányos Jedlik" };  
Artigo carro { gasolina };
```

```
// criado automaticamente pelo compilador  
Artigo::Artigo(const Artigo& param)  
{  
    titulo = param.titulo;  
    autor  = param.autor;  
}
```

Utiliza a atribuição
definida para as classes
String e string



Retorno de Objetos

- Ao retornar um objeto, **temos opções**:

- Uma referência ou uma cópia

```
String & operator=(const String & s);
```

```
String operator+(const String & s1, const String & s2);
```

- Uma referência constante ou cópia constante

```
const String & Maior(const String & s1, const String & s2);
```

```
const String Maior(const String & s1, const String & s2);
```

Retorno de Objetos

- É mais eficiente retornar uma referência constante
 - Se o retorno é um objeto passado:
 - Por parâmetro
 - Por invocação do objeto

```
String Maior(const String & s1, const String & s2)
{ return s1.Size() > s2.Size() ? s1 : s2; }
```

```
const String & Maior(const String & s1, const String & s2)
{ return s1.Size() > s2.Size() ? s1 : s2; }
```

```
const String & String::Maior(const String & s)
{ return Size() > s.Size() ? *this : s; }
```


Retorno de Objetos

- Para **objetos locais** a **métodos** ou funções
 - Não se deve retornar referências
 - Deve-se retornar uma cópia
 - Objeto local é destruído

```
String operator+(const String& s1, const String &s2)
{
    String s; // objeto será destruído após retorno

    s.tam = s1.tam + s2.tam;
    s.str = new char[s.tam+1]{0};
    memcpy(s.str, s1.str, s1.tam);
    memcpy(s.str + s1.tam, s2.str, s2.tam);
    return s;
}
```

Retorno de Objetos

- Uma **referência não-constante** é usada:

- Quando o retorno pode ser modificado

- Operador de atribuição =
- Operador de inserção <<

```
String & String::operator=(const String & s);  
String lang { "C++" };  
String prog;  
prog = lang;
```

```
ostream & operator<<(ostream & os, const String & s);  
cout << prog << lang;
```

Ponteiros para Objetos

- Ponteiros podem apontar para objetos

```
String frase[10];
```

```
for (int i = 0; i < 10; i++)  
    cin >> frase[i];
```

- Existentes:

```
String * primeira = &frase[0]; // frase
```

- Alocados dinamicamente:

```
String * favorita = new String { frase[4] };
```

Ponteiros para Objetos

- A **alocação dinâmica** invoca um construtor
 - Assim como a alocação automática

```
String texto;                                // construtor padrão
String frase { "Quem não tem cão" };        // construtor
String copia { frase };                      // construtor de cópia
```

```
String * txt = new String;                   // construtor padrão
String * str = new String { "Caça com gato" }; // construtor
String * cpy = new String { *str };           // construtor de cópia
```

→ O operador **new** é usado em **dois níveis**:
para alocar o objeto String e para alocar
o vetor de caracteres dentro da String

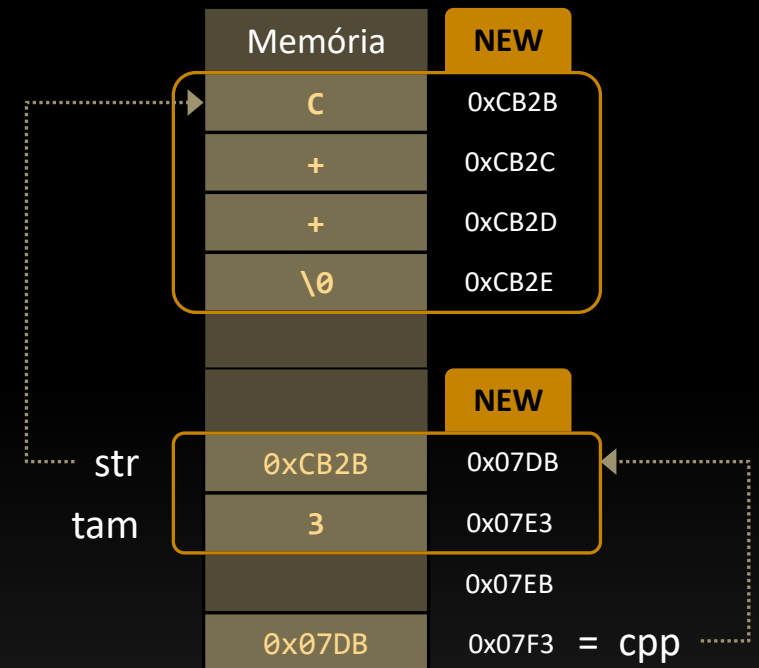
Ponteiros para Objetos

- Visualizando a **alocação do objeto**

```
String * cpp = new String { "C++" };
```

```
class String
{
private:
    char * str;
    size_t tam;
    ...
};
```

```
String::String(const char txt[])
{
    tam = strlen(txt);
    str = new char[tam + 1];
    strcpy(str, txt);
}
```



Ponteiros para Objetos

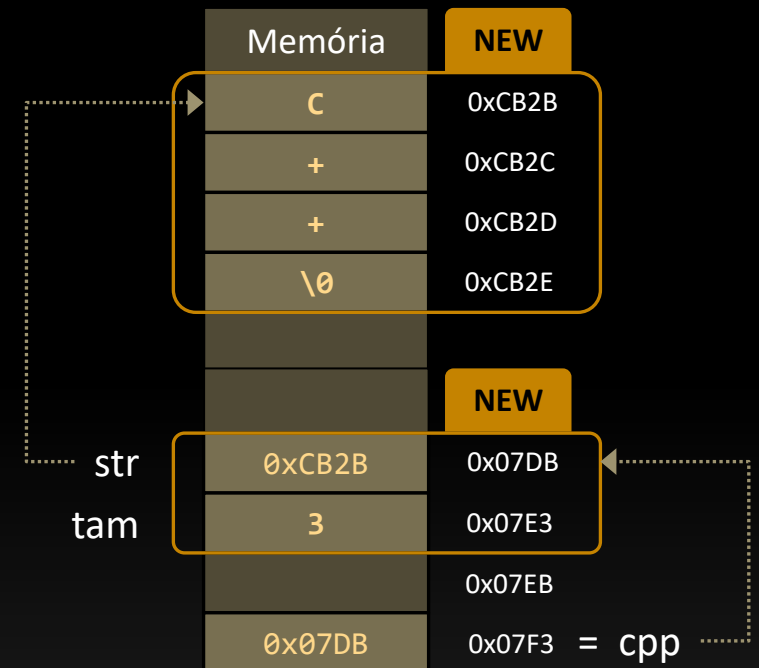
- Visualizando a **liberação do objeto**

```
String * cpp = new String { "C++" };
```

```
delete cpp;
```

```
class String
{
private:
    char * str;
    size_t tam;
    ...
};
```

```
// destrutor
String::~~String()
{
    delete [] str;
}
```



Exemplo

```
#include <iostream>
#include "String.h"
using namespace std;

int main()
{
    cout << "Entre com até 10 dizeres populares <linha vazia para sair>:\n";

    String frase[10];
    int i = 0;
    do
    {
        cout << i + 1 << ": ";
        cin >> frase[i];
    }
    while (frase[i++][0] != '\0' && i < 10);

    int total = i - 1;

    ...
}
```

Exemplo

```
if (total > 0)
{
    cout << "\nAqui estão os dizeres:\n";
    for (i = 0; i < total; i++)
        cout << frase[i] << endl;

    String * menor = &frase[0];
    for (i = 1; i < total; i++) {
        if (frase[i].Size() < menor->Size())
            menor = &frase[i];
    }
    cout << "\nFrase mais curta:\n" << *menor << "\n\n";

    String * favorita = new String{ frase[total-1] };
    cout << "Minha frase favorita:\n" << *favorita << endl;
    delete favorita;
}
else
    cout << "Nenhuma entrada!\n";
}
```


Exemplo

- Saída do programa:

```
Entre com até 10 dizeres populares <linha vazia para sair>:
```

```
1: Cão que ladra não morde
```

```
2: Santo de casa não faz milagre
```

```
3:
```

```
Aqui estão os dizeres:
```

```
Cão que ladra não morde
```

```
Santo de casa não faz milagre
```

```
Frase mais curta:
```

```
Cão que ladra não morde
```

```
Minha frase favorita:
```

```
Santo de casa não faz milagre
```

Resumo

- **Objetos** podem **alocar memória**
 - Eles são usados como outro objeto qualquer
 - A classe encapsula a complexidade
- **Objetos** podem **ser retornados**
 - Por referência: maior eficiência
 - Por cópia: quando necessário
- **Objetos** podem **ser alocados** com new
 - Um construtor é chamado
 - O **delete** deve ser usado